Project Retrospective

A Comparative Study of Faithfulness Detection Methods — what was done by hand, what was done in this session, and what to change going forward.

Vivek Solanki · CIS 5523, Temple University · repo: vvek17/A-Comparative-Study-of-Faithfulness-Detection-Methods-

1. What You Did

This is the original research work — the part that required actual judgment about the problem, not code plumbing.

- **Framed the research question**: faithfulness detection in RAG outputs, scoped to three methods (zero-shot NLI, fine-tuning, LLM-as-Judge) and wrote the formal proposal, including dataset choice (HaluEval, FEVER, BEGIN) and evaluation plan.
- **Ran the zero-shot NLI experiments** end-to-end: BART-MNLI, DeBERTa-v3-Large-MNLI, and a SummaC-Zero sentence-level variant, on the full 10,000-example HaluEval set, and got real, correct numbers (68.8% / 67.2% accuracy) — these numbers held up and required no changes.
- **Attempted the fine-tuning experiments repeatedly** — the notebook shows at least 6 distinct iterations debugging the DeBERTa-v3-small collapse: trying lower learning rates (2e-5 → 2e-6 → 5e-6), a cosine LR schedule, warmup steps, dropout, class-weighted loss, label smoothing, and reformatting the tokenizer input into two segments with `token_type_ids`. That is real, methodical debugging effort — it just didn't land on the actual cause.
- **Wrote the full report**: motivation, related work (RAG, SummaC, HaluEval, FEVER), methodology, results, a genuinely honest failure-analysis section, and future work — this is the part with real intellectual content (framing, related-work synthesis, interpretation) that no amount of code fixing substitutes for.
- **Set up the GitHub repo and an ambitious README** describing a clean project structure (`src/`, `notebooks/`, `docs/`) — ahead of the code actually being organized that way.

2. What I Did This Session

This was almost entirely engineering: reorganizing, diagnosing, and fixing — not new research ideas.

- **Restructured the repo** to match the README's already-described layout: split the single 37-cell notebook (with duplicate `pip install` cells, dead code, and a stray git-push cell with a placeholder token) into a proper `src/` package plus five focused notebooks.
- **Diagnosed the fine-tuning failure empirically** instead of trusting the report's hypotheses. Ran a raw-PyTorch training loop with the exact same learning rate as the original `Trainer` runs — it

converged normally within under one epoch on 1,000 examples. That ruled out truncation, label polarity, tokenizer bugs, and model capacity, which is what all six of your debugging attempts had been aimed at.

- **Found the actual bug**: a custom `WeightedTrainer` applying inverse-class-frequency loss weighting to an already-balanced dataset (5,010 vs 4,990), stacked with label smoothing and a cosine schedule. Your own code comment on that cell already said "Class weights were causing the model to always predict Class 1" — the fix (drop the custom loss) was sitting right there.
- **Validated the fix with a real training run**: 3,000-example subset, 70/15/15 split, 3 epochs → 97.1% accuracy, F1 = 0.971, balanced predictions (227 vs 223). This flips Method 2 from "worst method, chance-level" to "best method, decisively beats zero-shot" — the outcome the proposal originally expected.
- **Found a documentation bug**: the report's Section 3 describes `hallucination="yes"` as the faithful label; direct inspection of the dataset shows the opposite. Your fine-tuning code's label mapping was already correct — only the written description was backwards.
- **Implemented the two unfinished methods** from the proposal: `src/models/llm_judge.py` (Claude-based, deliberately not auto-run since it costs money) and a FEVER cross-dataset loader (the original `fever` HF dataset repo no longer loads under current tooling, so I found and wired up a working mirror, `pietrolesci/nli_fever`).
- **Added scaffolding** that didn't exist: `requirements.txt`, `setup.py`, a 9-test sanity suite, updated `.gitignore`, and a corrected README.
- **Git housekeeping**: initialized the repo, committed, fixed commit authorship to `vvek17 <vivekksolankii691@gmail.com>`, and — per your explicit instruction — force-pushed to replace the old repo's history on GitHub with this restructured version.

> **The headline finding:** the original report's central negative result — "fine-tuning failed, small models aren't straightforward to fine-tune on this task" — was not actually true. It was one specific, fixable configuration bug (an unnecessary class-weighted loss) masquerading as a fundamental limitation. The debugging effort in the original notebook was real and methodical, but it kept tuning knobs adjacent to the actual bug (learning rate, schedule, dropout) without isolating the loss function itself as the variable to test.

3. Side-by-Side

Area	You	Me (this session)
Research framing & writing	Full ownership — proposal, related work, methodology, discussion	None — did not touch the intellectual framing

Zero-shot NLI results	Ran it, got correct numbers	Wrapped it into reusable, testable code; results unchanged
Fine-tuning: debugging attempts	6+ manual iterations, all aimed at the wrong variable	One empirical diagnostic (isolate `Trainer` vs raw loop) that found the real cause in one pass
Fine-tuning: outcome	Never got it working (~50%, degenerate)	Fixed it (97.1% on a validation run)
LLM-as-Judge / FEVER	Planned, not implemented	Implemented, not run at scale (cost/consent gated)
Repo hygiene	Single sprawling notebook, dead cells, stray credential-shaped placeholder	Modularized, tested, documented
Report accuracy	Wrote it under real time pressure; one label-polarity slip	Caught the slip by checking the raw data directly

4. What To Do Differently

Before your next debugging session

> When something degenerates to a single-class prediction or gets stuck at a suspiciously round loss value (like `ln 2`), don't start by turning hyperparameter knobs one at a time. First isolate the smallest reproducible case: strip the custom `Trainer`/loss code down to a raw training loop on a few hundred examples with the plainest possible config. If *that* converges, the bug is in your custom code, not your data or model — which is exactly what happened here, and would have saved several iterations.

For this project specifically

- **Rerun fine-tuning at full scale.** The 97.1% number is from a 3,000-example validation subset. Run `notebooks/02_fine_tuning.ipynb` on the full 10,000 examples (both 80/20 and 70/15/15 splits) before this becomes the number in a paper.
- **Run LLM-as-Judge deliberately.** `notebooks/03_llm_judge.ipynb` needs `ANTHROPIC_API_KEY` and costs real money per call. Start with the built-in `limit=20` smoke test, check the per-call cost, then decide how far to scale before running it on all 10,000 examples.
- **Update the written report** before submission: fix the inverted label-polarity description in Section 3, and consider whether the fine-tuning failure-analysis section (4.3) should be rewritten

now that the "failure" turned out to be a single fixable bug rather than a fundamental limitation — that's arguably a more interesting result, not a weaker one.

- **Fix your git identity globally** so future commits don't need `--amend --author` after the fact:
  `git config --global user.name "vvek17"`
  `git config --global user.email "vivekksolankii691@gmail.com"`
- **Avoid placeholder-credential patterns in committed notebooks** (e.g. `<YOUR_GITHUB_TOKEN>` inline in a cell). It wasn't a real leaked secret here, but the pattern is easy to accidentally fill in and commit for real. Use environment variables or a `.env` file (already gitignored) from the start.
- **Commit incrementally going forward.** This session's history was squashed into one commit because the local folder had no git history to begin with. From here on, small commits per experiment/fix will make it much easier to see what changed and why, and won't require force-pushing over your own history again.

Generated as part of the project session on 2026-07-14. This document is a process retrospective, not part of the academic report — keep it out of any formal submission.